A Reliable Non-Intrusive Runtime Verification Framework for Linearizability

Abstract

Linearizability is the standard correctness condition for concurrent data structures. Even when an algorithm has been proved correct theoretically, translating it into code may introduce subtle errors that are difficult to detect. Existing runtime verification tools rely on intrusive instrumentation that introduces false positives and false negatives, as the observed execution may differ from the actual one.

This paper presents an open-source verification framework for runtime verification of linearizability. The framework treats the system as a black box, requiring only the class name of the concurrent implementation. It consists of two layers: an instrumentation layer, that provides two algorithms based on non-linearizable snapshot objects that obtain the current execution without modifying the system under inspection; and a monitoring layer, which guarantees that if the observed execution is not linearizable, the real execution is also not linearizable. When linearizable, the framework produces a witness — the best achievable given the impossibility result of [2].

The framework is implemented in Java and Clojure, supports queues, deques, sets, and maps, and can be extended to other data structures by providing their sequential specification. It is intended for developers and researchers who wish to validate that a concurrent implementation matches its theoretical design.

Keywords: Concurrent data structures, Distributed instrumentation, Linearizability, Runtime verification, Software tool, Verification framework

1. Introduction

Runtime Verification (RV) verifies correctness properties of the current execution of a system [12]. In this work, we focus on verifying *linearizability*

of concurrent data structures at runtime. Linearizability is the standard correctness condition for concurrent objects: informally, every operation must appear to take effect instantaneously at some point between its invocation and its response [10]. Even when an algorithm has been proved correct theoretically, its implementation may introduce subtle races that violate linearizability [16, 13].

Runtime verification of linearizability involves two tasks. The first is to *detect* the current execution: since operations may overlap, executions are naturally described as partial orders of invocation and response events [10]. Existing RV tools detect the current execution through intrusive instrumentation techniques such as bytecode weaving (e.g., AspectJ); however, the observed execution may differ from the one that would occur without instrumentation [6], introducing false positives and false negatives and making the verdict unreliable. The second task is to *decide* whether the detected execution is linearizable, which is NP-complete [8].

Contribution. We present an open-source verification framework that verifies the current execution without altering the system under inspection. The framework [3] treats the concurrent implementation as a black box, requiring only its class name.

The instrumentation layer provides two distributed algorithms for detection. The first, CollectRAW, is based on a non-linearizable snapshot object that utilizes an iterative collect object [15]; the second, CollectFAInc, induces an order via atomic fetch-and-increment. Both obtain the current execution asynchronously and guarantee that if the collected execution is not linearizable, the real execution is also not linearizable — providing a sound verdict and avoiding the false negatives introduced by intrusive instrumentation [6]. For the decision task, the monitoring layer invokes a JIT-based linearizability checker [14], producing a witness when the execution is linearizable.

The framework is implemented in Java with snapshot components in Clojure. It supports queues, deques, sets, and maps, and can be extended to other data structures by providing their sequential specification. It is intended for developers and researchers who wish to validate that a concurrent implementation matches its theoretical design. The software is publicly available and supports reproducible experiments.

2. Software Overview

The framework addresses two independent tasks: *detecting* the current execution of the system under inspection, and *deciding* whether that execution is linearizable. Figure 1 shows how the components interact to carry out these tasks.

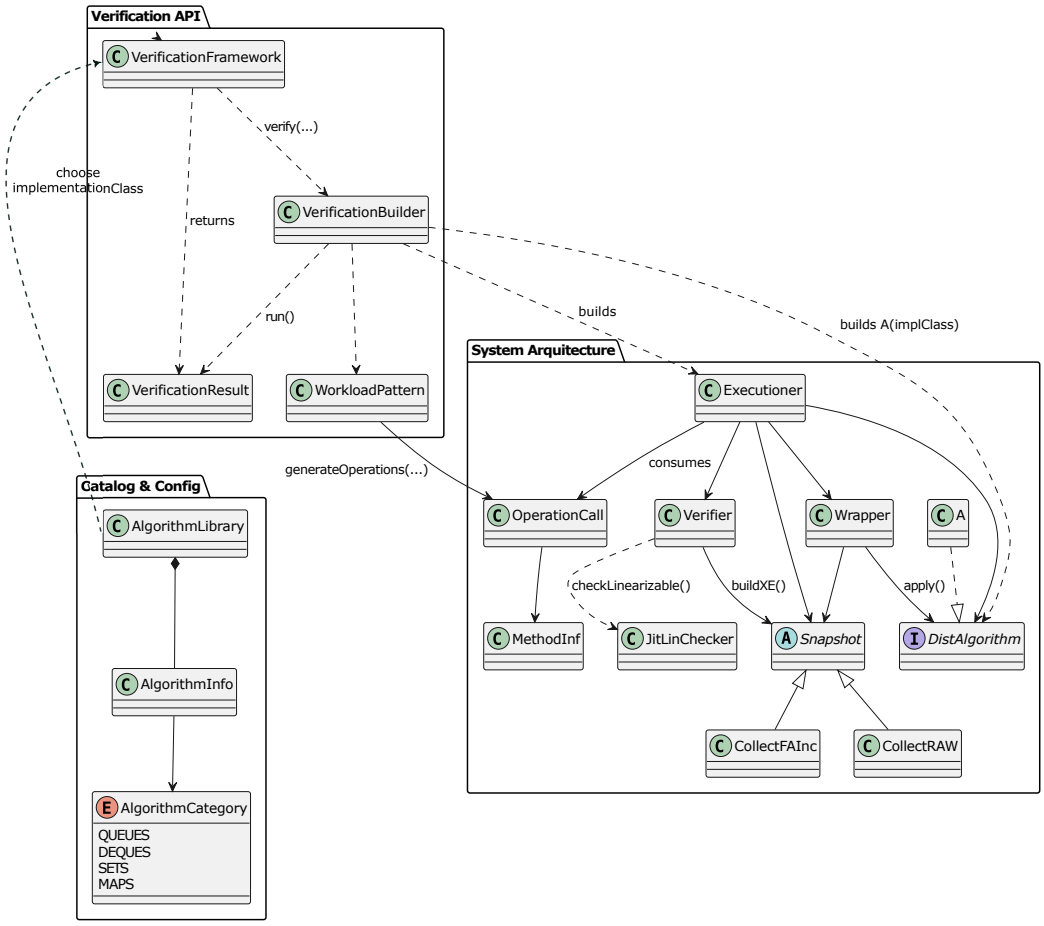

Figure 1: Interaction between the framework components during a verification run.

At the highest level, a user selects a concurrent implementation via the `VerificationFramework` API, which constructs an execution environment and returns a `VerificationResult`. Internally, the framework is organized in two layers: an *instrumentation layer* responsible for detecting the execution, and a *monitoring layer* responsible for deciding linearizability.

2.1. Instrumentation Layer

The instrumentation layer intercepts every operation invocation and response without modifying the implementation under inspection. It consists of the `Wrapper` and the `Snapshot` object.

The Wrapper (Algorithm 1) is a concurrent algorithm where worker threads record invocation and response events for each operation. These events are written to a shared `Snapshot` object, following the protocol in [2].

Algorithm 1 Wrapper: distributed instrumentation mechanism

```
procedure EXECUTE(op, params)
    id ← ThreadId()
    C.WRITE(id, ⟨invoke, op, params⟩)
    res ← A.APPLY(op, params)
    C.SNAPSHOT(id, ⟨return, res⟩)
end procedure
```

Snapshot objects.. The framework provides two non-linearizable collect implementations that share a common interface: `write`, `snapshot`, `buildXE` (Algorithms 2 and 3). Both are implemented in Clojure.

CollectFAInc (identifier `gAIsnap` in the repository) induces a total order on all events using a single shared atomic fetch-and-increment counter. Every call to `write` or `snapshot` atomically increments the counter and stores the resulting timestamp alongside the event in the invoking thread's local log (Algorithm 2). Because the counter is strictly monotone and the increment is atomic, timestamps are globally unique and totally ordered across threads. At the end of the execution, `buildXE` merges all per-thread logs, sorts them by timestamp, and returns the resulting sequence as X_E.

Algorithm 2 CollectFAInc: write and snapshot

```
procedure WRITE(id, ⟨invoke, op, params⟩)
    cnt ← fetch-and-increment(counter)
    opId ← ⟨id, localIndex[id]++⟩
    lastOpId[id] ← opId
    log[id].append({ :type invoke, :op-id opId, :tid id, :op op, :arg params, :count cnt})
end procedure
procedure SNAPSHOT(id, ⟨return, res⟩)
    cnt ← fetch-and-increment(counter)
    opId ← lastOpId[id]
    log[id].append({ :type return, :op-id opId, :tid id, :res res, :count cnt})
end procedure
procedure BUILDXE
    events ← ⋃_i log[i]
    return sort(events, by (:count, :tid))
end procedure
```

CollectRAW in Algorithm 3 (identifier `rawsnap` in the repository) maintains two separate per-thread logs: $invs[id]$ for invocations and $rets[id]$ for responses. Snapshot reads the current invocation logs of *all* threads and stores this cross-thread view alongside the response. BuildXE reconstructs the happens-before relation from the captured views: if $opId_k$ appears in the view of ret_i, then inv_k started before ret_i completed; if $view(i) \subsetneq view(j)$, then ret_i completed before inv_j started. The resulting partial order is obtained by a topological sort (Kahn's algorithm).

The invocation logs are read in Snapshot without additional synchronization — no `volatile` fields, no locks, no `ConcurrentArrayList`. This is intentional and corresponds to the theoretical model of a *non-linearizable* collect object [15]: the view captured at a response may miss invocations that are concurrently being written. A partial view makes the observed execution *more concurrent* than the real one (fewer precedence edges are established), never less. Soundness is preserved: if the observed execution — with potentially more concurrency — is not linearizable, then the real execution is also not linearizable.

Algorithm 3 CollectRAW: write, snapshot, and buildXE

```
procedure Write(id, ⟨invoke, op, params⟩)
    opId ← ⟨id, localIndex[id]++⟩
    lastOpId[id] ← opId
    invs[id].append({ :type invoke, :op-id opId, :tid id, :op op, :arg params})
end procedure
procedure Snapshot(id, ⟨return, res⟩)
    view ← copy(invs[0], ..., invs[n−1])                    ▷ snapshot of all invocations
    opId ← lastOpId[id]
    rets[id].append({ :type return, :op-id opId, :tid id, :res res, :view view})
end procedure
procedure BuildXE
    Build edges of a partial order:
        (a) inv_x → ret_x for each complete operation x
        (b) ret_i → inv_j if view(i) ⊊ view(j) and opId_j ∉ view(i)   ▷ ret_i completed before inv_j started
        (c) inv_k → ret_i if opId_k ∈ view(i) and k ≠ i               ▷ inv_k started before ret_i completed
    return topological-sort(edges)
end procedure
```

2.2. Monitoring Layer

After the instrumentation layer finishes collecting events, the monitoring layer reconstructs a well-formed execution X_E and decides its linearizability.

The `Verifier` shares the `Snapshot` object with the `Wrapper`. It calls `buildXE()` to obtain a well-formed execution (`IPersistentVector`) of events and passes it to the JIT-based linearizability checker (`JitLinChecker`) [14].

If the observed execution is *not* linearizable, the framework guarantees that the real execution is also not linearizable — a soundness property inherited from the non-linearizable collect objects [15]. The converse direction is not guaranteed, which is unavoidable given the impossibility result of [3].

2.3. Execution Environment

The `Executioner` instantiates the `Snapshot`, `Wrapper`, and `Verifier` components, spawns worker threads that invoke operations on the `DistAlgorithm` object, and collects the final verdict and execution time.

`DistAlgorithm` is an abstraction layer for the concurrent implementation under inspection. It wraps a Java concurrent class via reflection, exposing a uniform `apply(method, args)` interface that allows the framework to handle any data structure without modifying its source code.

The `VerificationFramework` API (Figure 1) exposes this execution environment to users through a builder pattern: the user selects a concurrent class from the `AlgorithmLibrary` (which catalogs queues, deques, sets, and maps), configures thread count and snapshot strategy, and calls `run()` to obtain a `VerificationResult`.

3. Experimental Evaluation

We evaluate the framework along two axes: instrumentation overhead and verdict accuracy. We follow the methodology of Georges et al. [7]: five warmup rounds (discarded) followed by ten measured rounds per cell; we report mean execution time in milliseconds. All experiments were run on a 64-core Linux server (OpenJDK 21.0.10, `-Xmx16g`). The AspectJ variant uses load-time weaving via `-javaagent:aspectjweaver`. Figure 2 summarizes the main results.

3.1. Instrumentation Overhead

Tables 1 and 2 measure `taskProducers()` time only — no linearizability checking is performed. All experiments use `java.util.concurrent.ConcurrentLinkedQueue` with operations `offer` and `poll`.

`CollectFAInc` and `CollectRAW` show virtually identical overhead across all configurations (difference <1% in all cells). Both scale linearly with operation count: increasing ops by 10× yields approximately 10× more time

Table 1: Instrumentation overhead vs. operation count (mean ms, threads = 4).

Ops	`CollectFAInc`	`CollectRAW`	AspectJ
100	0	2	2
500	9	9	7
1 000	20	21	16
5 000	101	101	79
10 000	204	204	160
50 000	1 031	1 028	869
100 000	2 005	2 002	1 757

Table 2: Instrumentation overhead vs. thread count (mean ms, ops = 10 000).

Threads	`CollectFAInc`	`CollectRAW`	AspectJ
2	160	159	133
4	204	204	160
8	373	379	250
16	476	482	378
32	560	563	439
64	608	593	507

(e.g., 20 ms at 1 000 ops to 2 005 ms at 100 000 ops with 4 threads). Scaling with thread count is sub-linear: going from 2 to 64 threads (32×) increases overhead by approximately 4× at 10 000 operations, indicating that the framework distributes work effectively across cores.

AspectJ is consistently 20–30% faster in raw instrumentation time because `@AfterReturning` records a single event per operation after completion, whereas the snapshot implementations record two events (invocation and response) with shared-state access in between. This performance gap does not reflect correctness, as shown next.

3.2. Verdict Accuracy on a Correct Implementation

Table 3 reports the percentage of correct verdicts over 100 complete pipeline runs per cell (instrumentation + verification, 100 operations). Since `ConcurrentLinkedQueue` is a correct implementation, the expected verdict is always *linearizable* (%T = 100, %F = 0, %E = 0).

`CollectFAInc` and `CollectRAW` produce correct verdicts in all 100 runs across all thread configurations. AspectJ produced false negatives in 55% of runs with 2 threads, decreasing monotonically to 0% at 32 and 64 threads. This pattern reveals two regimes. Under *high CPU contention* (thread count $<$ available cores), threads compete for CPU time and context switches between an operation's completion and its trace capture in `@AfterReturning` are frequent, producing observed executions that differ from the actual order. Under *genuine parallelism* (thread count $\geq$ cores), each thread executes on

Table 3: Verdict accuracy over 100 runs (ops = 100, `ConcurrentLinkedQueue`). %T = linearizable, %F = not linearizable, %E = error.

Thr.	`CollectFAInc`			`CollectRAW`			AspectJ		
	%T	%F	%E	%T	%F	%E	%T	%F	%E
2	100	0	0	100	0	0	45	55	0
4	100	0	0	100	0	0	79	21	0
8	100	0	0	100	0	0	98	2	0
16	100	0	0	100	0	0	99	1	0
32	100	0	0	100	0	0	100	0	0
64	100	0	0	100	0	0	100	0	0

a dedicated core without preemption, reducing the non-atomicity window to near zero. This confirms the structural nature of the observer effect identified by El-Hokayem and Falcone [6].

Comparison with El-Hokayem and Falcone [6].. Their work identifies AspectJ bytecode weaving as the source of false positives and false negatives in runtime verification, showing that the observed execution may differ from the actual one due to the non-atomicity of before/after instrumentation points. We use `@AfterReturning` as a *baseline instrumentation strategy*, isolating the instrumentation component from the verification component: all three strategies use the same JIT-Lin checker [14], so any difference in verdict accuracy is attributable solely to instrumentation. Their reference experiment reports false negative rates of 92–99% for AspectJ-based tools on an unsynchronized implementation. Our results are complementary: using a *correct* implementation, AspectJ still produces false negatives (up to 55%), while `CollectFAInc` and `CollectRAW` produce 0% in all configurations.

A natural question is whether `@Around` advice — which wraps the entire operation in a single control block — would close this reliability gap. We leave a systematic comparison as future work; we note, however, that `@Around` reduces but does not eliminate the non-atomicity window, and provides no formal soundness guarantee analogous to that of the collect objects [15].

3.3. Verdict Accuracy on Non-Linearizable Implementations

Severely broken implementation.. Table 4 reports verdict accuracy for `BrokenQueue`, an intentionally non-linearizable queue with a severe race condition. The expected verdict is always *not linearizable* (%T = 0, %F = 100, %E = 0).

All three strategies correctly identify `BrokenQueue` as non-linearizable in 100% of runs. The asymmetry with Table 3 is the key finding: AspectJ-based

Table 4: Verdict accuracy over 100 runs (ops = 100, `BrokenQueue`, expected: not linearizable).

Thr.	`CollectFAInc`			`CollectRAW`			AspectJ		
	%T	%F	%E	%T	%F	%E	%T	%F	%E
2	0	100	0	0	100	0	0	100	0
4	0	100	0	0	100	0	0	100	0
8	0	100	0	0	100	0	0	100	0
16	0	100	0	0	100	0	0	100	0
32	0	100	0	0	100	0	0	100	0
64	0	100	0	0	100	0	0	100	0

instrumentation produces false negatives exclusively on *correct* implementations — precisely the scenario where unreliable verdicts are most harmful, as a developer receiving a spurious non-linearizable verdict may incorrectly modify code that does not need to change.

Subtly broken implementation.. Table 5 reports verdict accuracy for `PartialSyncQueue`, an implementation with a subtle race condition between `isEmpty()` and `removeFirst()` in `poll()`: `offer()` is synchronized but `poll()` is not, so a second thread may empty the queue between the `isEmpty()` check and the `removeFirst()` call. Unlike `BrokenQueue`, this violation manifests only under specific interleavings and does not occur in every execution.

Table 5: Verdict accuracy over 100 runs (ops = 60, `PartialSyncQueue`). %T = linearizable, %F = not linearizable, %E = error.

Thr.	`CollectFAInc`			`CollectRAW`			AspectJ		
	%T	%F	%E	%T	%F	%E	%T	%F	%E
2	91	9	0	95	5	0	36	64	0
4	85	15	0	96	4	0	67	33	0
8	84	16	0	92	8	0	98	2	0
16	97	3	0	94	6	0	100	0	0
32	100	0	0	99	1	0	100	0	0
64	100	0	0	99	1	0	100	0	0

The %F column for `CollectFAInc` and `CollectRAW` reflects the fraction of runs in which the race condition manifested and was correctly detected (3–16% depending on thread count). The remaining runs correspond to executions where the violation did not occur within 60 operations and the execution was genuinely linearizable — a correct verdict in both cases. No false positives are produced: every non-linearizable verdict corresponds to a real violation. AspectJ shows the opposite pattern at low thread counts: with 2 threads, it produces 64% false negatives — runs where the violation occurred but the non-atomic instrumentation failed to capture it, caus-

ing the verifier to see a linearizable trace. The contrast between Tables 4 and 5 illustrates the key distinction between detection rate and soundness: `CollectFAInc` and `CollectRAW` are sound in both cases, whether the implementation is severely broken or only subtly so.

3.4. Coverage Across Data Structure Types

Table 6 evaluates the framework across all supported data structure types (threads = 4, ops = 30, 10 runs per cell) on the 64-core Linux server. Results were consistent with a separate run on a 10-core macOS laptop, confirming that the observed behavior is independent of the hardware platform.

Table 6: Coverage across data structure types (threads = 4, ops = 30, 10 runs). OK = verdict matches expected; FAIL = unexpected verdict.

Implementation	**Exp.**	`CollectFAInc`		`CollectRAW`		**AspectJ**
		%T	Match	%T	Match	Match
Queues						
ConcurrentLinkedQueue	LIN	100	OK	100	OK	OK (60%T)
LinkedBlockingQueue	LIN	100	OK	100	OK	OK (70%T)
LinkedTransferQueue	LIN	100	OK	100	OK	OK (60%T)
Deques						
ConcurrentLinkedDeque	LIN	100	OK	100	OK	OK (60%T)
LinkedBlockingDeque	LIN	100	OK	100	OK	OK (90%T)
Sets						
ConcurrentSkipListSet	LIN	100	OK	100	OK	OK (100%T)
Maps						
ConcurrentHashMap	LIN	100	OK	100	OK	**FAIL (0%T)**
ConcurrentSkipListMap	LIN	100	OK	100	OK	**FAIL (0%T)**
Non-linearizable implementations						
BrokenQueue	NOT	0	OK	0	OK	OK (0%T)
NonLinearizableQueue	NOT	10	OK	10	OK	OK (10%T)
PartialSyncQueue	NOT	90	OK	95	OK	OK (60%T)

`CollectFAInc` and `CollectRAW` produce correct verdicts for all 11 implementations. AspectJ produces false negatives in queue and deque implementations (10–40%) and fails entirely on map implementations, reporting 100% false negatives for both `ConcurrentHashMap` and `ConcurrentSkipListMap`. This failure on maps is consistent across both test machines and is attributable to incorrect argument capture in `@AfterReturning` for multi-argument operations such as `put(key, value)`: the resulting trace is structurally malformed for the verifier, independently of the thread scheduler.

Note on sample size.. Due to the NP-completeness of linearizability checking [8], verdict experiments are limited to 60–100 operations and 100 runs per cell, except Table 6 which uses 10 runs per cell. The observed trends are consistent with the theoretical predictions of [6].

4. Usage and Availability

The framework is implemented in Java and Clojure, built with Maven, and distributed under the Apache 2.0 license at:

`https://github.com/PRISM-Concurrent/efficient-distributed-rv`

Minimal usage example.. To verify a `ConcurrentLinkedQueue` with 4 threads and 100 operations, a user writes:

```
VerificationResult result = VerificationFramework
    .forClass("java.util.concurrent.ConcurrentLinkedQueue")
    .withThreads(4)
    .withOperations(100)
    .withSnapshot("CollectFAInc")
    .verify();
System.out.println(result.isLinearizable());
```

No modifications to the implementation under test are required. The framework can be extended to other data structures by providing a sequential specification in Clojure following the examples in `src/main/clojure/spec/`.

Reproducing the experiments.. The results reported in Tables 1–3 can be reproduced with:

```
mvn clean package -DskipTests
java -cp target/efficient-distributed-rv-*-jar-with-dependencies.jar \
     phd.experiments.VerifierBenchmark --format=org \
     --output=results/verifier.org
```

The repository includes detailed documentation (`README.md`, `QUICK_START.md`), example workloads, and configuration files.

5. Related Work

Several tools perform runtime verification for concurrent Java programs using intrusive instrumentation. Java-MOP [4], Tracematches [1], and LARVA [5] generate monitors from formal specifications and rely on AspectJ for instrumentation. As shown experimentally by El-Hokayem and Falcone [6], this approach introduces false positives and false negatives because the observed

execution may differ from the actual one — a problem our framework avoids by design. These tools also focus on sequential properties expressed in total-order formalisms (LTL, FSMs, regular expressions), whereas linearizability is inherently a property of partial orders.

Tools such as JPaX [9] and RVPredict [11] target specific concurrency errors — data race freedom and deadlock freedom — rather than linearizability. Their underlying models (lockset analysis, maximal causal models) are well-suited to those properties but do not generalize to linearizability checking.

The closest related work on the verification side is the JIT-based linearizability checker of Lowe [14], which our framework integrates in the monitoring layer. Lowe's tool addresses the decision problem but assumes the execution has already been correctly captured; it does not address the instrumentation problem. Our framework complements this checker with a non-intrusive instrumentation layer that provides the soundness guarantee absent from AspectJ-based approaches.

Software Metadata

Item	Description
Software name	Distributed Runtime Verification Framework of Linearizability
Version	v1.0.0
Release date	2025-12-15
Authors	Anonymous (to be filled after review)
Repository (public)	`https://github.com/PRISM-Concurrent/efficient-distributed-rv`
Persistent identifier (DOI)	To be assigned
License	Apache License, Version 2.0 (Apache-2.0)
Programming language	Java, Clojure
Build system	Maven
Operating systems	Linux, macOS, Windows
Requirements	JDK 21+, Maven 3.9+, Clojure runtime
Documentation	`README.md`, `INSTALL.md`, `API_USAGE_GUIDE.org`, `USER_MANUAL.md`, `API_EXAMPLES.md`, `QUICK_START.md`
Support email	Available after acceptance

Table 7: Software metadata required for an Original Software Publication.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process. During the preparation of this work, the authors used Claude in order to assist with language editing, grammar, style revision, and LaTeXformatting of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Declaration of interests. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Bodden, E., Hendren, L., Lam, P., Lhoták, O., Naeem, N.A., 2010. Collaborative runtime verification with tracematches. Journal of Logic and Computation 20, 707–723. doi:`10.1093/logcom/exn077`.

[2] Castañeda, A., Rodríguez, G.V., 2023. Asynchronous wait-free runtime verification and enforcement of linearizability, in: Proceedings of the 2023 ACM Symposium on Principles of Distributed Computing, Association for Computing Machinery, New York, NY, USA. p. 90–101. URL: https://doi.org/10.1145/3583668.3594563, doi:10.1145/3583668.3594563.

[3] Castañeda, A., Rodríguez, G.V., 2026. Asynchronous wait-free runtime verification and enforcement of linearizability. J. ACM 73. URL: https://doi.org/10.1145/3777409, doi:10.1145/3777409.

[4] Chen, F., Roşu, G., 2005. Java-MOP: A monitoring oriented programming environment for Java, in: Tools and Algorithms for the Construction and Analysis of Systems (TACAS), Springer. pp. 546–550. doi:10.1007/978-3-540-31980-1_36.

[5] Colombo, C., Pace, G.J., Schneider, G., 2009. LARVA — Safer monitoring of real-time Java programs, in: 7th IEEE International Conference on Software Engineering and Formal Methods (SEFM), IEEE Computer Society. pp. 33–37. doi:10.1109/SEFM.2009.13.

[6] El-Hokayem, A., Falcone, Y., 2018. Can we monitor all multithreaded programs?, in: Colombo, C., Leucker, M. (Eds.), Runtime Verification, Springer International Publishing, Cham. pp. 64–89.

[7] Georges, A., Buytaert, D., Eeckhout, L., 2007. Statistically rigorous Java performance evaluation, in: Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA), ACM. pp. 57–76. doi:10.1145/1297027.1297033.

[8] Gibbons, P.B., Korach, E., 1997. Testing shared memories. SIAM Journal on Computing 26, 1208–1244. doi:10.1137/S0097539794279614.

[9] Havelund, K., Roşu, G., 2004. An overview of the runtime verification tool Java PathExplorer. Formal Methods in System Design 24, 189–215. doi:10.1023/B:FORM.0000017721.39909.4b.

[10] Herlihy, M.P., Wing, J.M., 1990. Linearizability: a correctness condition for concurrent objects. ACM Trans. Program. Lang. Syst. 12, 463–492.

URL: https://doi.org/10.1145/78969.78972, doi:10.1145/78969.78972.

[11] Huang, J., Meredith, P.O., Roşu, G., 2014. Maximal sound predictive race detection with control flow abstraction, in: Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), ACM. pp. 337–348. doi:10.1145/2594291.2594315.

[12] Leucker, M., Schallhart, C., 2009. A brief account of runtime verification. Journal of Logic and Algebraic Programming 78, 293–303. doi:10.1016/j.jlap.2008.08.004.

[13] Lourenço, J.M., Fiedor, J., Křena, B., Vojnar, T., 2018. Discovering concurrency errors, in: Verification and Assurance of Concurrent Software. Springer. volume 10691 of *Lecture Notes in Computer Science*, pp. 34–82. doi:10.1007/978-3-319-75632-5_2.

[14] Lowe, G., 2017. Testing for linearizability. Concurrency and Computation: Practice and Experience 29, e3928. doi:https://doi.org/10.1002/cpe.3928. e3928 cpe.3928.

[15] Rodríguez, G.V., Castañeda, A., 2024. Towards efficient runtime verified linearizable algorithms, in: Ábrahám, E., Abbas, H. (Eds.), Runtime Verification, Springer Nature Switzerland, Cham. pp. 262–281.

[16] Treiber, R.K., 1986. Systems Programming: Coping with Parallelism. Technical Report RJ 5118. IBM Almaden Research Center.

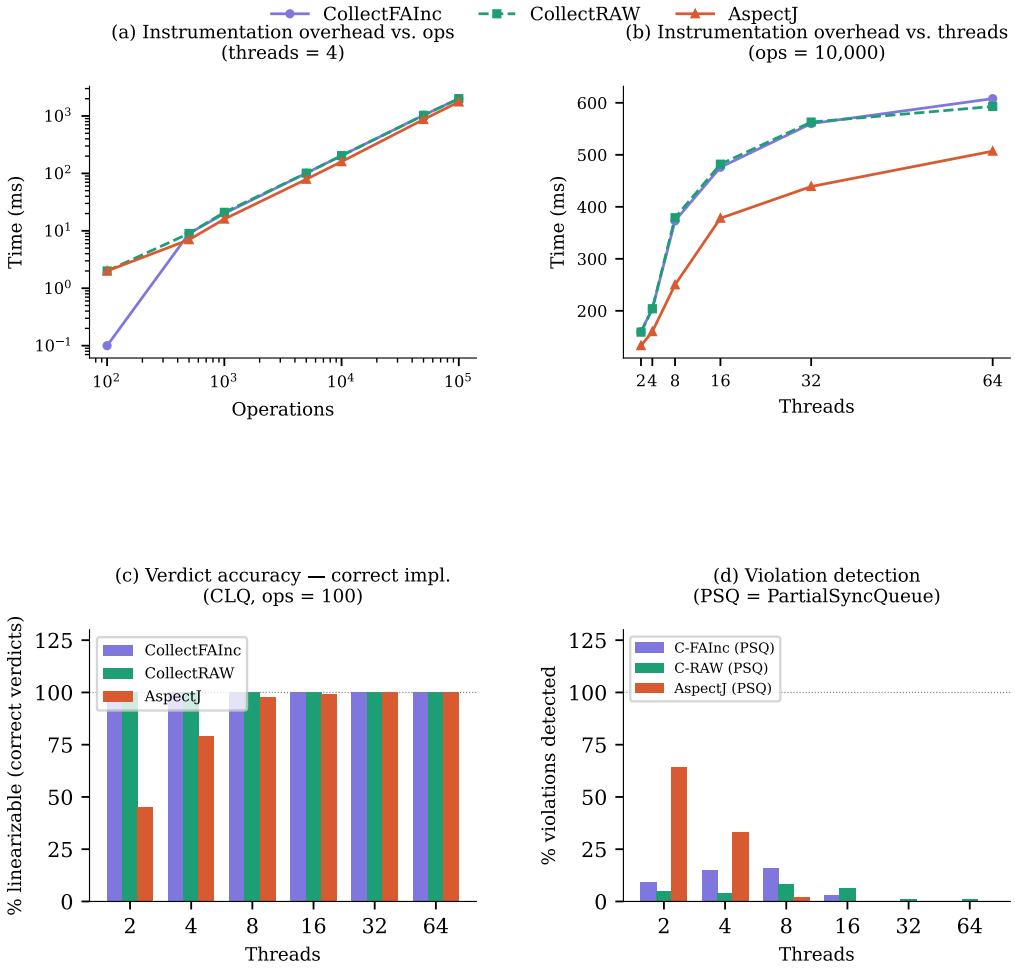

Figure 2: Summary of experimental results. **(a)** Instrumentation overhead vs. operation count (threads = 4, log–log scale). **(b)** Instrumentation overhead vs. thread count (ops = 10 000). **(c)** Verdict accuracy on a correct implementation (`ConcurrentLinkedQueue`, ops = 100, 100 runs per cell): `CollectFAInc` and `CollectRAW` achieve 100% correct verdicts at all thread counts; AspectJ degrades to 45% under contention. **(d)** Violation detection rate for `BrokenQueue` (BQ, severe race condition) and `PartialSyncQueue` (PSQ, subtle race condition). All strategies detect BQ in 100% of runs; PSQ detection varies with the manifestation rate of the race condition.